MTH 4300
Spring 2026

Programming Problem Set 3: Arrays and Pointers - arrayFunctions.cpp

Directions: Download the template files provided on Brightspace. Submit your source code via Gradescope, in `.cpp` format.

Submission Reminder: Submitting for feedback is optional. If you submit by the deadline and want feedback beyond the autograder, indicate that in your submission. You may resubmit unlimited times.
Important: Passing all test cases does not guarantee correctness. The autograder tests limited inputs. Test additional cases yourself.
Note: While learning, avoid IDE tools that autocomplete or generate code for you (e.g., Copilot, IntelliSense suggestions). Writing code yourself builds the understanding you'll need for exams.

Working with AI on This Assignment

This assignment has a structured AI component. Follow these steps in order.

Step 1: Write your code on your own first. Give each function a genuine attempt before using any AI. If your code compiles and runs but gives wrong answers, try tracing through it with a specific input on paper before moving on to Step 2.

Step 2: If you're stuck, use AI as a debugging tutor. Don't paste your code and ask "what's wrong." Instead, use a prompt like this:

> *"I'm writing a C++ function that [describe what it should do]. Here is my code. I'm getting [wrong output / crash / compiler error]. Don't fix my code or show me corrected code. Instead, pick a specific line and ask me what I think a particular variable points to at that moment. Help me find the bug by making me trace through my own logic. Keep a back-and-forth conversation."*

If you use this step, save the conversation — you'll submit a link to it.

Step 3: After your code passes the autograder, do a code review conversation. The goal is to test whether you can explain what you wrote. Paste one of the prompts below along with your working code into a new AI conversation (ChatGPT, Claude, Copilot, etc.). The AI will quiz you — treat it like a study session. This is especially valuable for pointers and memory management, where it is common to have code that works without fully understanding *why* it works.

For `intersection`, use this prompt:

> *"Here is my working C++ code for the intersection function. Give me a specific pair of arrays and ask me to trace through my code step by step: which elements get compared, in what order, and what the count is at each stage. After I trace it, tell me if I got it right. Then give me a trickier case (e.g., empty array, no overlap, full overlap). Do 3–4 rounds, getting progressively harder. Don't explain my code to me — make me explain it to you."*

For `insert`, use this prompt:

> *"Here is my working C++ code for the insert function. Act as a technical interviewer. Ask me questions one at a time about my code. Start with what specific lines do, then build toward harder questions about why I made certain choices and what would go wrong if I changed something. If my answer is vague or wrong, don't correct me — ask a follow-up that helps me see the issue. Do 5–6 rounds."*

Submit a link to each conversation (one per function) as a comment at the top of your `.cpp` file, along with any Step 2 conversation links.

Be sure to read the SPECIFICATIONS carefully! And write comments!

General Specifications and Notes:

- No global variables should be used.
- You can only use the header `iostream`.
- You cannot use vectors or any advanced data structure that can store an arbitrarily long collection of integers, other than built-in C++ arrays.

Note: Each problem has test cases on Gradescope. All test cases are visible, so you can see exactly which cases your code passes or fails. Use this feedback to improve your solution. You may resubmit as many times as you like. Remember, these assignments are for practice; use them to prepare for exams!

The autograder will call your functions directly and check the return values. You do not need to handle any input or output — just implement the functions as specified.

Recall that built-in types, such as numbers and characters, are defined directly by C++, and that arrays are included in these types. These are types that represent facilities that are provided by most computer hardware. The standard library defines additional types that are higher-level that the computer hardware does not implement directly. So, arrays in C++ are not equipped with directly available functions for many operations you may have been familiar with from Python lists. In this problem, you will write some functions that can be used to perform some basic operations on arrays.

In this assignment you will implement the functions `intersection` and `insert` described below that operate on arrays. The interface for the functions should not be modified. You are given a `.h` file with function declarations (you will not submit this or modify it). There is a separate template `.cpp` file where you will write your function implementations.

You can write whatever you want to test your functions in a `main` function in your program; however, **the arrayFunctions.cpp file in your submission to Gradescope should NOT have a `main` function**.

3a) `intersection`

Implement the function with the declaration

```
int intersection(int* , int, int* , int);
```

The function takes two arrays and their lengths as arguments:

- The first argument is a pointer to the first element of an array of integers.
- The second argument is that array's length.
- The third argument is a pointer to the first element of a second array of integers.
- The fourth argument is the length of the second array.

Your function should count how many values appear in *both* arrays and return that count.

You can assume that the lengths are passed in correctly, and that neither array contains repeated elements.

Hint: For each element in one array, check whether it appears anywhere in the other array. A nested loop is a natural approach here.

For example, if the first array contains 1, 2, 3 and the second contains just 3, the function should return 1. Code that calls the function could look like:

```
int a1[3] = {1, 2, 3};
int a2[1] = {3};
int n = intersection(a1, 3, a2, 1);
```

The integer `n` would have the value 1 because the two arrays share one value (they both have 3).
Here is another example. If the first array contains 5, 10, 15, 20 and the second contains 10, 30, 20, 40:

```
int a1[4] = {5, 10, 15, 20};
int a2[4] = {10, 30, 20, 40};
int n = intersection(a1, 4, a2, 4);
```

The integer `n` would have the value 2 because the arrays share two values (10 and 20).

3b) `insert`

Implement the function with the declaration

```
void insert(int** , int, int, int);
```

The function takes four arguments:

- The first argument is *a pointer to a pointer* to the first element of a dynamically allocated array of integers.
- The second argument is the current length of the array.
- The third argument is the integer value to insert.
- The fourth argument is the index at which to insert it.

The function does not return anything. Instead, it should create a new array (one element larger), copy the elements over with the new value inserted at the given index, and update the pointer so it points to the new array. The remaining elements after the insertion point should shift down by one position.

If the index is out of bounds (negative, or greater than the array's length), the function should do nothing. Inserting at the start (index 0) or right after the last element (index equal to the length) is valid. You can assume the correct length will be passed in.

Remember to free any memory that is no longer in use!

Important: This function assumes the array was allocated on the heap (using `new`). Do not pass in a stack-allocated array. The function will call `delete[]` on the old array, which is only valid for heap-allocated memory.

Hint: Think about why we need a pointer to a pointer here. The function needs to change which array the caller's pointer points to. If we only received `int*`, we would get a copy of the pointer, and the caller would never see the new array.

For example, if an array contains 1, 2, 3 and we insert 7 at index 2, the resulting array should be 1, 2, 7, 3. If we then insert 8 at index 4, the resulting array should be 1, 2, 7, 3, 8. Code that calls the function could look like:

```
int* a1 = new int[3];
a1[0] = 1; a1[1] = 2; a1[2] = 3;
int** ptrA1 = &a1;
insert(ptrA1, 3, 7, 2);
// a1 now points to {1, 2, 7, 3}
insert(ptrA1, 4, 8, 4);
// a1 now points to {1, 2, 7, 3, 8}
```

3c) **AI Code Review + Practice Questions**

AI Code Review: After your code passes the autograder, complete the two AI code review conversations described at the beginning of this assignment (one for `intersection`, one for `insert`). Submit a link to each conversation as a comment at the top of your `.cpp` file. See the "Working with AI on This Assignment" box on page 1 for the prompts and full instructions.

The following are extra practice questions for you. They are good preparation for exams. Try them on your own, and let me know if you have questions.

1. In part (a), what happens if you call the function with an incorrect length (either longer or shorter than the actual array) for one or both of the arrays? Try it and describe what you observe.
2. In part (a), does it matter if the arrays passed in as arguments are allocated on the stack or the heap? Why or why not?
3. In part (b), what would happen if the first parameter were `int*` instead of `int**`? Why wouldn't that work?
4. Go through all of the variables in your `insert` implementation, including the input parameters. Which ones are on the stack? Which are on the heap?